

Flujo completo: crear un estudiante nuevo

Proyecto TiConvivencia — Frontend (Angular) + Backend (Node/Express) + Base de datos (MySQL)
Generado el 12 de agosto de 2026 · Guía de estudio personal

Contenido

- 1. El estudiante de ejemplo: Juan Pérez
- 2. Diagrama general del recorrido
- 3. FRONTEND — paso a paso (archivo por archivo)
- 4. BACKEND — paso a paso (archivo por archivo)
- 5. BASE DE DATOS — tablas, Primary Keys y Foreign Keys
- 6. Glosario de términos usados

1. El estudiante de ejemplo

Para seguir el código con un caso concreto, usamos estos datos de ejemplo que el usuario escribe en el formulario:

Campo del formulario	Valor de ejemplo
RUN	12345678
DV (dígito verificador)	9
Nombre	Juan
Apellido	Pérez
Sexo	M
Curso	3° Básico A (internamente <code>id_curso = 5</code>)

Este objeto en el navegador se ve así (en TypeScript, dentro del componente):

```
form = {
  run: "12345678",
  dv: "9",
  nombre: "Juan",
  apellido: "Pérez",
  sexo: "M",
  id_curso: 5
}
```

2. Diagrama general del recorrido



3. FRONTEND Angular

Carpeta: `front-ticonvivencia/src/app/`

1 El usuario hace clic en "Agregar"

Archivo: `features/estudiantes/estudiantes.html` , línea 8

```
<button (click)="abrirForm()">Agregar</button>
```

Esto ejecuta `abrirForm()` en `estudiantes.ts` , líneas 60–78 . Como no se pasa ningún estudiante existente, entra al bloque `else` (línea 74) y deja el formulario vacío — esto es lo que activa el "modo crear" en vez de "modo editar".

2 El usuario llena el formulario

Archivo: `estudiantes.html` , líneas 100–141

Cada input usa `[(ngModel)]` para conectarse a una propiedad del objeto `form` definido en `estudiantes.ts` , líneas 33–40 .

Analogía: el objeto `form` es como una ficha de papel con casillas (RUN, DV, Nombre...). Cada input HTML es un lápiz conectado por un "cable invisible" a su casilla. Lo que escribes en el input queda guardado solo en la variable — y si el código cambia la variable, el input se actualiza solo. Por eso, al editar un estudiante existente (línea 65-72), basta con asignar `this.form = {...}` y los inputs se llenan solos.

```
<input [(ngModel)]="form.nombre" name="nombre" />
<input [(ngModel)]="form.run" name="run" />
<select [(ngModel)]="form.id_curso" name="curso">...</select>
```

3 El usuario hace clic en "Agregar estudiante" (submit)

Archivo: `estudiantes.html` , línea 146 → dispara `guardar()`

Función `guardar()` en `estudiantes.ts` , líneas 84–107 :

```
guardar() {
  const { run, dv, nombre, apellido, sexo, id_curso } = this.form; // línea 86

  if (!run || !dv || !nombre || !apellido || !sexo || !id_curso) { // línea 88
    this.error.set('Complete todos los campos requeridos');
    return; // corta aquí si falla
  }

  const request = this.editando()
    ? this.api.updateEstudiante(...)
    : this.api.createEstudiante(this.form); // línea 95 – CREAR (nuestro caso)

  request.subscribe({
    next: () => { /* mensaje éxito, cierra modal, recarga lista */ }, // líneas 98–101
    error: (err) => { this.error.set(err.error?.message); }
  });
}
```

Con nuestro Juan Pérez, como es un estudiante nuevo (`editando()` es `null`), se ejecuta la línea 95: `this.api.createEstudiante(this.form)` .

4 Se arma y envía la petición HTTP (POST)

Archivo: `core/services/api.services.ts` , líneas 34–36

```
createEstudiante(data: any) {
  return this.http.post(`${this.base}/estudiantes`, data);
}
```

`this.base` viene de `environments/environment.ts` , línea 3 : `apiUrl: 'http://localhost:3000/api'` .

Analogía del sobre: un POST es como enviar una carta por correo. La URL (`http://localhost:3000/api/estudiantes`) es la dirección del destinatario, y el objeto `data` (nuestro `form` con los datos de Juan) es la carta metida dentro del sobre.

Petición real que sale del navegador:

```
POST http://localhost:3000/api/estudiantes
Body: { "run":"12345678", "dv":"9", "nombre":"Juan",
        "apellido":"Pérez", "sexo":"M", "id_curso":5 }
```

4. BACKEND Node / Express

Carpeta: `backticonvivencia/src/` (repo hermano, aparte del frontend)

5 El servidor recibe la petición y la dirige

Archivo: `index.js`, línea 13

```
app.use('/api/estudiantes', require('./routes/estudiantes.routes'));
```

Traducción: "todo lo que llegue a la dirección `/api/estudiantes`, entrégaselo al archivo de rutas de estudiantes".

6 Pasa por dos "filtros de seguridad" (middlewares)

Archivo: `routes/estudiantes.routes.js`

```
router.use(verifyToken); // línea 5

router.post('/', requireRole('ENCARGADO'), create); // línea 10
```

- `verifyToken` (`middleware/auth.js`, líneas 3–16): revisa que el usuario tenga sesión iniciada (token JWT válido). Si no, corta con error `401`.
- `requireRole('ENCARGADO')` (`middleware/auth.js`, líneas 19–23): exige que el usuario logueado tenga el rol `ENCARGADO`. Si un profesor sin ese rol intenta crear un estudiante, corta con error `403` — nunca llega al siguiente paso.

Por qué importa: estos filtros son la razón por la que no cualquiera puede crear estudiantes solo porque conoce la URL de la API — hace falta estar logueado Y tener el rol correcto.

7 El controlador valida y guarda

Archivo: `controllers/estudiantes.controller.js`, función `create`, líneas 19–37

```
const create = async (req, res) => {
  const { run, dv, nombre, apellido, sexo, id_curso } = req.body; // línea 20

  if (!run || !dv || !nombre || !apellido || !sexo || !id_curso) // línea 22
    return res.status(400).json({ message: 'Complete todos los campos requeridos' });

  const [result] = await pool.query( // línea 26
    `INSERT INTO ESTUDIANTE (run, dv, nombre, apellido, sexo, id_curso, id_establecimiento)
    VALUES (?, ?, ?, ?, ?, ?, ?)`,
    [run, dv, nombre, apellido, sexo, id_curso, req.user.id_establecimiento] // línea 28
  );

  res.status(201).json({ id_estudiante: result.insertId, message: 'Estudiante creado' });
};
```

Dato clave: el backend vuelve a validar los campos (líneas 22-23) aunque el frontend ya lo hizo. Nunca hay que confiar solo en la validación del navegador — alguien podría llamar a la API directamente sin pasar por el formulario.

¿De dónde sale `id_establecimiento` si el formulario no lo pide? Sale del token de sesión del usuario logueado (`req.user.id_establecimiento`), no del formulario. Así, cada colegio (establecimiento) solo puede crear y ver sus propios estudiantes — un usuario del Colegio A nunca ve ni crea estudiantes del Colegio B, aunque ambos usen la misma base de datos. Esto se llama **multi-tenant** (múltiples "inquilinos" comparten el mismo sistema pero sus datos están aislados).

Si el RUN ya existe, MySQL avisa con el código `ER_DUP_ENTRY` y el backend responde `409` (líneas 33-34) — esto confirma que existe una restricción de unicidad sobre esa columna en la base de datos.

5. BASE DE DATOS MySQL

Nota honesta: el esquema exacto de la tabla `ESTUDIANTE` (tipos de columna, si el `UNIQUE` es sobre `run` o sobre `run+dv`, etc.) no está guardado como archivo `.sql` en el repo — ya quedó anotado como pendiente en el `TODO` del backend: "confirmar con `SHOW CREATE TABLE ESTUDIANTE;` cuando haya acceso a la BD". Lo que sigue está deducido leyendo cómo el código usa la tabla — es confiable, pero conviene confirmarlo cuando tengas acceso directo a MySQL.

8 Estructura deducida de la tabla ESTUDIANTE

```
ESTUDIANTE
├ id_estudiante      PK (autoincremental, MySQL lo genera solo)
├ run
├ dv
├ nombre
├ apellido
├ sexo
├ activo             (true/false – se usa en toggleActivo)
├ id_curso           FK → CURSO.id_curso
└ id_establecimiento FK → ESTABLECIMIENTO.id_establecimiento
```

¿Qué es una PRIMARY KEY (PK)?

Es el "documento de identidad" único de cada fila de la tabla. Nunca se repite y nunca se elige a mano: MySQL lo genera solo, sumando 1 al último usado. Por eso el `INSERT` del paso 7 (línea 27-28) **no** incluye `id_estudiante` — y por eso MySQL lo devuelve de vuelta en `result.insertId` (línea 31), que es como decir "listo, guardé a Juan Pérez y le asigné el número 42".

¿Qué es una FOREIGN KEY (FK)?

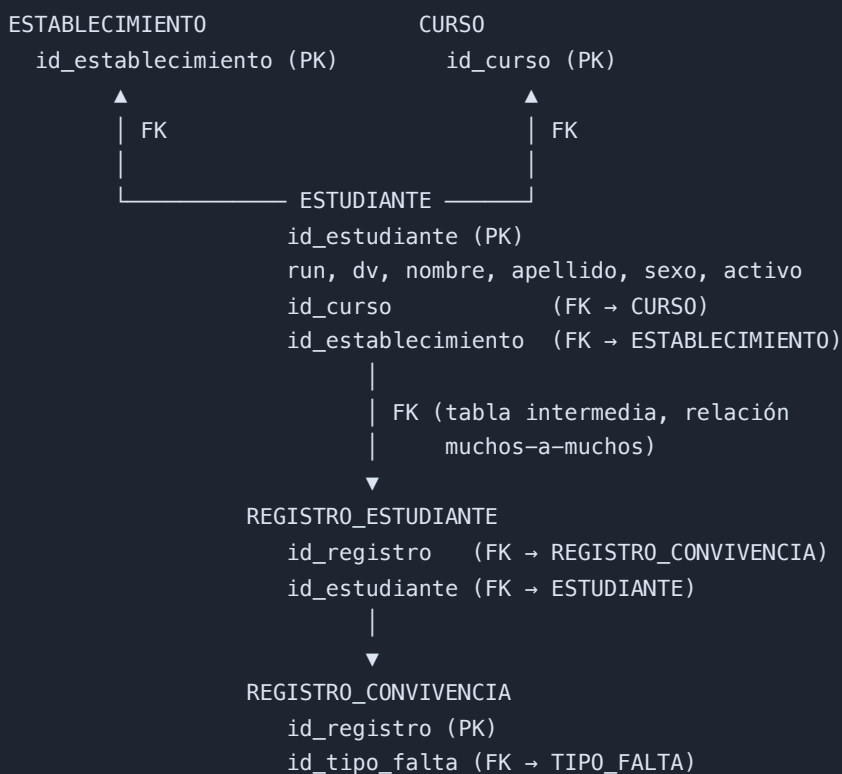
Es un "puntero" desde una tabla hacia la fila de otra tabla, en vez de repetir la información completa.

Analogía: en vez de escribir "3° Básico A" a mano en cada fila de estudiante que pertenece a ese curso (y arriesgarte a escribirlo distinto cada vez: "3ero basico A", "3° BASICO A"...), guardas solo el número `id_curso = 5`, que apunta a la única fila de la tabla `CURSO` donde vive ese nombre. Si mañana cambia el nombre del curso, lo cambias en un solo lugar y se actualiza para todos los estudiantes automáticamente.

Para volver a juntar el nombre del curso con cada estudiante en la pantalla de listado, el backend usa un `JOIN` — "pega" ambas tablas usando ese puntero:

```
SELECT e.*, c.nombre AS curso_nombre, c.grado
FROM ESTUDIANTE e
JOIN CURSO c ON e.id_curso = c.id_curso
WHERE e.id_establecimiento = ?
```

Relación completa entre tablas involucradas



Esta última parte explica por qué un estudiante puede tener **varios** incidentes de convivencia y un incidente puede involucrar a **varios** estudiantes: no se conectan directo, sino a través de la tabla intermedia **REGISTRO_ESTUDIANTE** (ver **estudiantes.controller.js**, líneas 113-121, función **consultarRut**). Esto se llama relación **muchos-a-muchos (N:M)**.

Otra pista del código: al intentar eliminar un estudiante (función **remove**, línea 77), si MySQL responde con el código **ER_ROW_IS_REFERENCED_2**, significa que ese estudiante tiene incidentes asociados en **REGISTRO_ESTUDIANTE** — la base de datos **rechaza el borrado** para no dejar registros huérfanos apuntando a un estudiante que ya no existe. Esto es una restricción **ON DELETE RESTRICT** en la Foreign Key.

6. Glosario rápido

Término	Qué significa en este proyecto
Componente (Angular)	Un archivo <code>.ts</code> + su <code>.html</code> que controla una pantalla o parte de ella (ej: la pantalla de Estudiantes)
<code>[(ngModel)]</code>	Conecta un input del HTML con una variable de TypeScript en ambas direcciones (binding bidireccional)
Service (<code>ApiService</code>)	Archivo centralizado que agrupa todas las llamadas HTTP hacia el backend, para no repetir código en cada componente
POST	Tipo de petición HTTP usada para crear algo nuevo en el servidor (a diferencia de GET, que solo consulta)
Middleware	Función que se ejecuta antes del controlador para revisar algo (sesión, permisos) y decidir si deja pasar la petición
JWT (token)	Un "carnet" digital que prueba que el usuario ya inició sesión, sin tener que mandar la contraseña en cada petición
Multi-tenant	Varios colegios (establecimientos) comparten el mismo sistema, pero cada uno solo ve y crea sus propios datos
Controlador	Función del backend que recibe la petición, valida los datos y decide qué hacer (guardar, actualizar, borrar)
Primary Key (PK)	Columna que identifica de forma única cada fila de una tabla
Foreign Key (FK)	Columna que apunta al PK de otra tabla, para relacionar información sin repetirla
JOIN	Instrucción SQL que junta dos tablas relacionadas por su FK, para traer datos combinados en una sola consulta
Relación N:M (muchos-a-muchos)	Cuando ambos lados de una relación pueden tener varios del otro lado — se resuelve con una tabla intermedia (ej: <code>REGISTRO_ESTUDIANTE</code>)

Generado a partir del código real de **front-ticonvivencia** y **backticonvivencia** el 12 de agosto de 2026.
Todas las rutas de archivo y números de línea fueron verificados leyendo el código fuente directamente.